

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
FACULDADE DE ENGENHARIA DE SOFTWARE**

EDUARDA LUIZA PINHEIRO NEPOMUCENO (24002529)

JEAN YUKI KIMURA (24008214)

JHENIFER LAÍS BARBOSA (24014979)

JOÃO PEDRO DUARTE GIATTI (24019083)

Sched — Documentação Final de Arquitetura

Disciplina: Padrões e Arquitetura de Software

CAMPINAS

2026

Contents

1	INTRODUÇÃO	1
1.1	Apresentação e justificativa	1
1.2	Objetivos	1
1.3	Problema e público-alvo	1
2	ATRIBUTOS DE QUALIDADE E DECISÕES ARQUITETURAIS	3
2.1	Atributos de qualidade prioritários (ISO/IEC 25010:2023)	3
2.2	Registro de Decisões Arquiteturais (ADRs)	4
2.2.1	ADR-001 — Monolito modular organizado em camadas	4
2.2.2	ADR-002 — Autenticação stateless via JWT	5
2.2.3	ADR-003 — Isolamento multi-tenant por <code>company_id</code>	5
2.2.4	ADR-004 — Strategy + Registry para regras de alerta	6
2.2.5	ADR-005 — Port/Adapter para o serviço de IA (anticorrupção)	6
2.2.6	ADR-006 — Exclusão lógica (soft delete)	7
2.2.7	ADR-007 — Provedor de usuário autenticado injetável (DIP)	7
3	ESTILO ARQUITETURAL	9
3.1	Plano macro — decomposição em unidades implantáveis	9
3.2	Plano interno — organização do código	9
4	APLICAÇÃO DOS PRINCÍPIOS SOLID	10
4.1	SRP — Single Responsibility Principle	10
4.2	OCP — Open/Closed Principle	10
4.3	LSP — Liskov Substitution Principle	10
4.4	ISP — Interface Segregation Principle	11
4.5	DIP — Dependency Inversion Principle	11
5	CLEAN CODE	12
6	PADRÕES DE PROJETO GOF	13
6.1	Strategy (Comportamental)	13
6.2	Adapter (Estrutural)	14
6.3	Builder (Criacional)	15
7	DESIGN DE API	16
7.1	Estilo e justificativa	16
7.2	Autenticação	16
7.3	Versionamento	16
7.4	Convenções de erro e paginação	16

7.5	Endpoints	17
7.6	Schemas principais	19
8	DIAGRAMAS E MODELOS	20
8.1	Arquitetura geral do sistema	20
8.2	Diagramas de classes dos padrões GoF	20
8.2.1	Padrão Strategy — motor de alertas	20
8.2.2	Padrão Adapter — integração com o serviço de IA	21
8.2.3	Padrão Builder — construção de entidades de domínio	22
8.3	Diagrama de sequência — previsão de demanda por IA	23
9	CONCLUSÃO	25
	REFERÊNCIAS	26
	ANEXOS	27

1 INTRODUÇÃO

1.1 Apresentação e justificativa

O **Sched** é uma plataforma de apoio à decisão para minimercados, padarias e pequenos comércios alimentares. Ele controla produtos, estoque (por lotes com validade), vendas e gera previsões de demanda baseadas em Machine Learning, com o objetivo de proteger o fluxo de caixa e reduzir o desperdício de alimentos. O backend já existia (originário do Projeto Integrador V) e foi aqui evoluído e organizado para evidenciar, de forma demonstrável, uma arquitetura justificada, baseada nos princípios SOLID, Clean Code e padrões GoF.

O escopo desta refatoração concentrou-se exclusivamente na pasta **api/** (backend desenvolvido em Spring Boot). O frontend, a pasta **ai/** (serviço de IA em Flask) e o banco de dados relacional foram totalmente preservados sem alterações estruturais.

1.2 Objetivos

- **Geral:** Prover uma API segura, extensível e manutenível para a gestão de estoque e vendas, contendo previsão de demanda integrada e isolamento lógico por empresa.
- **Específicos:**
 - Registrar produtos e lotes de estoque associados a datas de validade rígidas;
 - Registrar vendas com baixa automática de estoque baseada no algoritmo FIFO (*First-In, First-Out*), garantindo que lotes com vencimento mais próximo saiam primeiro;
 - Emitir alertas automatizados de vencimento próximo e de estoque baixo;
 - Integrar de forma desacoplada a um serviço externo de Inteligência Artificial para previsão de demanda e recomendação de reposição;
 - Garantir o isolamento estrito *multi-tenant*, onde cada empresa visualiza unicamente os seus próprios dados corporativos através de autenticação via tokens JWT.

1.3 Problema e público-alvo

Pequenos comércios alimentares operam historicamente com margens de lucro apertadas e um alto risco latente de perdas financeiras por vencimento de produtos. Atualmente, faltam ferramentas acessíveis no mercado que cruzem de forma simples os dados de estoque corrente, histórico de vendas e previsão preditiva de demanda.

O público-alvo deste sistema compreende proprietários e operadores de minimercados ou padarias que necessitam tomar decisões rápidas de compra e reposição. As regras de negócio não são triviais, pois envolvem o gerenciamento de filas FIFO para lotes, isolamento

multi-tenancy em nível de aplicação, segurança de dados comerciais, regras de alertas extensíveis e integração de rede com serviços externos distribuídos.

2 ATRIBUTOS DE QUALIDADE E DECISÕES ARQUITETURAIS

2.1 Atributos de qualidade prioritários (ISO/IEC 25010:2023)

1. Manutenibilidade (Modularidade, Testabilidade, Reusabilidade):

- *Justificativa:* Tratando-se de uma equipe de desenvolvimento pequena (4 membros) e de um sistema sob evolução contínua, o custo de engenharia dominante reside na capacidade de alterar o código com segurança e previsibilidade. A adição recorrente de novos tipos de alertas e a necessidade de isolar o serviço externo de IA sem quebrar o núcleo de negócio tornam a manutenibilidade o atributo de maior impacto prático no ciclo de vida do sistema.
- *Decisões arquiteturais:* Adoção de uma arquitetura em camadas combinada a um monolito modular (ADR-001); aplicação do padrão *Strategy* para o motor de alertas (ADR-004); implementação do padrão *Port/Adapter* para isolar o serviço de IA (ADR-005); e a substituição completa de utilitários estáticos pelo componente injetável `AuthenticatedUserProvider` (ADR-007).
- *Métricas observáveis:* Número de classes alteradas para a adição de um novo modelo de alerta (meta: 1 classe, atingida pelo uso do *Strategy*); cobertura de testes unitários concentrada nas classes de serviço (6 classes de teste estruturadas); monitoramento do acoplamento aferente e eferente por pacotes.

2. Segurança (Confidencialidade, Integridade, Autenticidade):

- *Justificativa:* O sistema armazena dados comerciais e financeiros altamente sensíveis de múltiplas empresas concorrentes em uma mesma base de dados, tornando o vazamento de informações entre *tenants* inaceitável tanto do ponto de vista legal quanto competitivo.
- *Decisões arquiteturais:* Utilização de arquitetura de segurança baseada em JWT *stateless* e criptografia BCrypt (ADR-002); aplicação de isolamento mandatório por `company_id` em todas as consultas SQL e validações de serviço (ADR-003); remoção total do segredo padrão inseguro da aplicação e fechamento de endpoints expostos da IA.
- *Métricas observáveis:* Número de endpoints expostos publicamente (restritos a apenas 3: `login`, criação de `company` e especificação aberta); ausência absoluta de segredos textuais em código fonte (*hardcoded*); testes automatizados específicos contra acesso cruzado de dados entre empresas distintas.

3. Confiabilidade (Maturidade, Tolerância a Falhas, Integridade de Dados):

- *Justificativa:* Falhas na baixa de estoques ou inconsistências de vendas corrompem diretamente o balanço financeiro e o planejamento logístico do comerciante. O

algoritmo FIFO, que percorre múltiplos lotes em uma única operação de venda, exige garantias transacionais rígidas para evitar estados inconsistentes no banco de dados.

- *Decisões arquiteturais:* Demarcação de todas as operações de escrita com a anotação `@Transactional` (garantindo atomicidade entre a baixa FIFO de múltiplos lotes e o registro da venda); implementação de exclusão lógica via *soft delete* para preservação de histórico analítico (ADR-006); validação rígida de saldo suficiente antes da persistência; tratamento centralizado através de um `GlobalExceptionHandler`; e degradação suave do adaptador de IA (nunca retornando `null`, mas sim coleções vazias).
- *Métricas observáveis:* Taxa de erros do tipo 5xx monitorados via Spring Boot Actuator; ausência histórica de registros com estoque negativo; consistência transacional validada por testes de estresse.

2.2 Registro de Decisões Arquiteturais (ADRs)

As decisões arquiteturais críticas foram registradas no formato canônico (NYGARD, 2011) e versionadas na pasta `/docs/adrs` do repositório. Os sete ADRs são apresentados na íntegra a seguir.

2.2.1 ADR-001 — Monolito modular organizado em camadas

Status: Aceito

Contexto: O Sched é mantido por uma equipe pequena (4 pessoas) e tem um domínio altamente coeso — produtos, estoque, vendas, alertas e previsão de demanda — de uma mesma empresa. Não há requisito de escala independente por subsistema nem times separados que justifiquem fronteiras de implantação distintas. Precisamos de uma arquitetura que favoreça manutenibilidade e velocidade de evolução sem custo operacional elevado.

Decisão: Adotar um monolito modular com arquitetura em camadas: `controller` → `service` → `record`, complementada pelos pacotes `dto`, `exception`, `security` e `client`. Cada camada conhece apenas a camada imediatamente inferior; entidades JPA não cruzam a fronteira do controller (são convertidas em DTOs do tipo `record`). A borda de integração externa (IA) usa o padrão porta/adaptador.

Consequências:

- + Baixa complexidade operacional: um único artefato implantável.
- + Limites de responsabilidade claros e testáveis por camada.
- + Refatoração interna barata, sem contratos de rede entre módulos.

- Escala apenas de forma vertical ou horizontal do monolito inteiro; subsistemas não escalam de forma independente. Aceitável para o volume atual.
- Microserviços foram explicitamente descartados por representarem *overengineering* frente ao tamanho do time e à coesão do domínio.

2.2.2 ADR-002 — Autenticação *stateless* via JWT

Status: Aceito

Contexto: O frontend é uma aplicação web separada que consome a API por HTTP. Não há estado de sessão a manter no servidor, e o sistema deve permitir escalabilidade horizontal sem necessidade de replicação de sessão entre instâncias.

Decisão: Usar autenticação *stateless* com JWT (*Bearer*). O `TokenService` (biblioteca `java-jwt`) emite tokens assinados com HMAC256 no login; o `SecurityFilter` (`OncePerRequestFilter`) valida o token a cada requisição e popula o `SecurityContext`. O Spring Security é configurado com `SessionCreationPolicy.STATELESS`, CSRF desabilitado e senhas em BCrypt. O segredo e a expiração são externalizados por variáveis de ambiente; não há segredo padrão — a ausência do segredo provoca falha na inicialização (*fail-fast*). Esta decisão foi reforçada na refatoração de 2026 com a remoção do fallback inseguro (`my-secret`).

Consequências:

- + Sem estado de sessão; qualquer instância valida qualquer token.
- + Segredo externalizado e expiração configurável por ambiente.
- Revogação de token antes da expiração exige mecanismo adicional (*blocklist*), ainda não implementado — registrado como dívida técnica.

2.2.3 ADR-003 — Isolamento *multi-tenant* por *company_id*

Status: Aceito

Contexto: Cada empresa cadastrada deve enxergar apenas seus próprios produtos, estoques, vendas e usuários. Vazamento de dados entre empresas é um risco de segurança e de confiabilidade inaceitável, tanto pelo aspecto legal quanto pelo competitivo.

Decisão: Adotar *multi-tenancy* lógico por discriminador (`company_id`). O usuário autenticado carrega sua `Company`; todas as consultas de leitura filtram por `product.company.id` ou `company.id` (ex.: `findByProduct_Company_IdAndProduct_DeletedFalse`), e as operações de escrita validam, na camada de serviço, se o recurso pertence à empresa do usuário, lançando `AccessDeniedException` em caso de violação.

Consequências:

- + Isolamento de dados garantido nas camadas de aplicação e de consulta.

- + Modelo simples, sem custo de bancos ou schemas separados por tenant.
- A correção do isolamento depende de disciplina na escrita dos services; mitigado por testes unitários que cobrem cenários de acesso cruzado entre empresas.
- Não há isolamento físico; um bug de query poderia vaziar dados — por isso os filtros por empresa são padronizados nos nomes de método dos repositórios.

2.2.4 ADR-004 — *Strategy + Registry para regras de alerta*

Status: Aceito

Contexto: O sistema gera alertas de estoque de naturezas diferentes (vencimento próximo, estoque baixo), com tendência de surgirem novos tipos (ruptura, excesso). Um bloco `if/else` ou `switch` por tipo no `AlertService` violaria o princípio Aberto-Fechado e tornaria a adição de novos alertas um risco de regressão.

Decisão: Modelar cada regra como uma *Strategy* implementando a interface `StockAlertRule` (métodos `type()` e `evaluate(companyId)`). O `AlertService` recebe por injeção a `List<StockAlertRule>` e monta um *Registry* (`Map<AlertType, StockAlertRule>`) indexado por `type()`. A seleção da regra é uma simples busca no mapa, sem condicionais.

Consequências:

- + Adicionar um novo tipo de alerta requer apenas a criação de uma nova classe `@Component` que implementa `StockAlertRule`; o `AlertService` não é modificado (OCP).
- + Cada regra é testável de forma completamente isolada.
- Mais classes pequenas no projeto — custo desprezível frente ao ganho de extensibilidade.

2.2.5 ADR-005 — *Port/Adapter para o serviço de IA (anticorrupção)*

Status: Aceito

Contexto: A previsão de demanda é fornecida por um serviço Python/Flask externo (pasta `ai/`), que não pode ser alterado nesta iteração e pode ser substituído no futuro por outro provedor. O backend não deve acoplar suas regras de negócio ao protocolo HTTP nem ao formato de contrato específico desse serviço.

Decisão: Definir uma porta `DemandForecastPort` (interface) no pacote `client` e um adaptador `FlaskDemandForecastClient` que implementa a porta por meio de `RestTemplate`. O `AIService` depende exclusivamente da porta (Inversão de Dependência); a URL do serviço é externalizada por `@Value` com padrão local configurável.

Consequências:

- + A substituição do provedor de IA requer apenas uma nova implementação da porta, sem nenhuma alteração no domínio.

- + Testes do `AIService` utilizam mock da porta, sem necessidade de rede.
- + A camada anticorrupção isola completamente o backend do contrato específico do Flask.
- Uma indireção adicional no código; justificada pela fronteira de integração externa.

2.2.6 ADR-006 — Exclusão lógica (*soft delete*)

Status: Aceito

Contexto: Produtos, usuários e empresas podem ser “removidos” da operação ativa, mas registros históricos — vendas e lotes de estoque — referenciam essas entidades. A exclusão física quebraria a integridade referencial e eliminaria o histórico necessário para relatórios analíticos e para o treinamento do modelo de previsão de demanda.

Decisão: Adotar *soft delete* com a flag booleana `deleted`. As consultas utilizam variações `...AndDeletedFalse`; a “remoção” apenas marca `deleted = true`. Regras de negócio protegem operações inválidas: a `ProductHasStockException` impede a exclusão de um produto com lotes de estoque ainda ativos.

Consequências:

- + Histórico preservado e integridade referencial mantida.
- + Possibilidade de auditoria e de reativação de registros.
- Todas as queries precisam filtrar `deleted = false`, com risco de esquecimento; mitigado por nomes de método padronizados nos repositórios Spring Data JPA.
- Volume de dados cresce sem mecanismo de expurgo automático; aceitável no volume atual de pequenos comércios.

2.2.7 ADR-007 — Provedor de usuário autenticado injetável (*DIP*)

Status: Aceito (refatoração 2026)

Contexto: O acesso ao usuário autenticado era realizado por um utilitário estático (`SecurityUtils.getAuthenticatedUser()`) chamado diretamente em todos os services. Isso acoplava cada service ao `SecurityContextHolder` do Spring Security, exigia mock estático nos testes (`mockStatic`) e violava o princípio da Inversão de Dependência, pois os módulos de alto nível dependiam de um detalhe de implementação concreto e estático.

Decisão: Substituir o utilitário estático por um colaborador injetável `Authenticated-UserProvider` (`@Component`) com o método `getCurrentUser()`. Os services passam a recebê-lo por injeção de construtor (via `@RequiredArgsConstructor`) e dependem dessa abstração. O `SecurityUtils` foi removido completamente da base de código.

Consequências:

- + Services desacoplados do acesso estático ao contexto de segurança (DIP aplicado).
- + Testes utilizam mock simples da interface (`@Mock`), sem necessidade de `mockStatic`.
- + Ponto único para evoluir a resolução do usuário (ex.: cache, leitura de claims adicionais).
- Uma dependência adicional no construtor de cada service; custo trivial.

A lógica de negócio (verificação de `deleted`, validação de posse por empresa) foi preservada nos services para manter o comportamento e a suíte de testes existente. Este ADR registra uma **decisão modificada**: a abordagem estática anterior foi substituída pela injetável após identificação do impacto negativo na testabilidade.

3 ESTILO ARQUITETURAL

3.1 Plano macro — decomposição em unidades implantáveis

O sistema adota o estilo de **monolito modular**, sendo compilado em um único artefato Spring Boot executável. Essa abordagem é justificada na ADR-001: as dimensões reduzidas da equipe, o domínio altamente coeso do sistema e a ausência inicial de requisitos de escalabilidade independente eliminam a necessidade de adoção de microsserviços, que adicionariam custos operacionais sem retorno financeiro imediato. A única fronteira distribuída da aplicação consiste no serviço externo de IA (Flask), o qual é consumido por meio do padrão *Port/Adapter* acoplado a protocolos HTTP/JSON, isolando o núcleo do negócio de mudanças de provedores tecnológicos.

3.2 Plano interno — organização do código

A estrutura interna baseia-se em uma **arquitetura em camadas** tradicional com forte isolamento e contornos hexagonais em suas fronteiras de comunicação externa:

- **controller:** Responsável pela adaptação do protocolo HTTP de entrada (`AuthController`, `ProductController`).
- **service:** Concentrador exclusivo das regras de negócio do domínio (`ProductService`, `SaleService`, `AlertService`).
- **repository:** Camada de acesso e persistência de dados gerenciada pelo Spring Data JPA (`ProductRepository`, `StockRepository`).
- **domain:** Modelagem de entidades ricas do sistema (`Company`, `User`, `Product`, `Stock`, `Sale`).
- **dto:** Estruturas de dados imutáveis (*records*) para contratos de entrada e saída.
- **security:** Mecanismos de criptografia, filtros de requisição de rede JWT e contexto (`SecurityConfig`, `AuthenticatedUserProvider`).
- **client:** Implementação da porta e adaptadores do ecossistema de Inteligência Artificial (`DemandForecastPort`, `FlaskDemandForecastClient`).
- **exception:** Centralização de erros e manipuladores globais (`GlobalExceptionHandler`).

4 APLICAÇÃO DOS PRINCÍPIOS SOLID

4.1 SRP — Single Responsibility Principle

Cada classe possui apenas uma única razão para ser alterada. O componente `AlertService`, por exemplo, atua exclusivamente selecionando e delegando tarefas; a responsabilidade de saber *como* processar e calcular cada alerta específico pertence unicamente às regras correspondentes injetadas.

```
@Service
public class AlertService {
    private final Map<AlertType, StockAlertRule> rulesByType;
    private final AuthenticatedUserProvider authenticatedUserProvider;

    private List<AlertResponse> evaluate(AlertType type) {
        User authUser = authenticatedUserProvider.getCurrentUser();
        Long companyId = authUser.getCompany().getId();
        return rulesByType.get(type).evaluate(companyId);
    }
}
```

4.2 OCP — Open/Closed Principle

A inclusão de novos tipos ou formatos de alertas no sistema **não exige nenhuma modificação** interna no código-fonte do componente `AlertService`. O sistema encontra-se aberto para extensão e fechado para modificação.

```
public interface StockAlertRule {
    AlertType type();
    List<AlertResponse> evaluate(Long companyId);
}

@Component
public class LowStockRule implements StockAlertRule {
    public AlertType type() { return AlertType.LOW_STOCK; }
    public List<AlertResponse> evaluate(Long companyId) { /* ... */ }
}
```

4.3 LSP — Liskov Substitution Principle

Qualquer classe que implemente a interface `DemandForecastPort` pode substituir o adaptador de produção sem causar quebras estruturais ou comportamentais indesejadas

ao `AIService`, dado que o contrato estabelecido assegura que coleções nulas nunca serão retornadas (degradando estritamente para listas vazias).

```
@Override
public List<AIPredictionResponse> forecast(List<AIDemandDataRequest>
    demandData) {
    AIPredictionResponse[] response = restTemplate.postForObject(forecastUrl,
        demandData, AIPredictionResponse[].class);
    return response == null ? List.of() : List.of(response);
}
```

4.4 ISP — Interface Segregation Principle

As interfaces expostas pelo sistema são altamente específicas e enxutas. A interface `StockAlertRule` declara unicamente os métodos `type()` e `evaluate()`, enquanto `DemandForecastPor` declara estritamente o método `forecast()`. Nenhum cliente do sistema é induzido a depender de métodos abstratos que não utiliza, simplificando a criação de dublês de testes (*mocks*).

4.5 DIP — Dependency Inversion Principle

Os módulos de alto nível não se acoplam a detalhes de baixo nível, mas dependem estritamente de abstrações. O `AIService` consome a porta aberta de previsão de demanda em vez do cliente HTTP direto (`RestTemplate`). Complementarmente, os serviços de negócio agora dependem da interface injetável `AuthenticatedUserProvider`, eliminando o acoplamento estático anterior ao contexto global do Spring Security.

5 CLEAN CODE

Diversas práticas de legibilidade e engenharia de software limpa foram introduzidas durante a refatoração do código-fonte do sistema:

- **Nomes claros e expressivos:** O método `getProductStockSummary()` substituiu o antigo termo genérico e semanticamente incorreto `findById()` exposto no `StockController`. Adicionalmente, a constante expressiva `Stock.NO_EXPIRATION` substituiu o uso disseminado de números mágicos temporais (ex: `LocalDateTime.of(1970,1,1,0,0)`).
- **Funções pequenas e coesas:** Métodos de serviços foram refatorados para manter baixa complexidade ciclomática, extraíndo conversões de dados para subfunções isoladas de mapeamento como `mapToResponse()` e `mapToBatchResponse()`.
- **Remoção de duplicidade:** Exclusão de consultas SQL redundantes em base de dados, como chamadas repetidas a `userRepository.findByEmail()` no interior do componente `SaleService`, visto que o principal já trafegava a entidade de usuário autenticada.
- **Tratamento explícito de erros:** Exceções de negócio customizadas (`InsufficientStockException`, `ProductHasStockException`, `ResourceNotFoundException`) passaram a ser interceptadas de forma centralizada pelo `GlobalExceptionHandler`, que agora formata respostas HTTP estruturadas devolvendo erros do tipo 400 (*Bad Request*) mapeando chaves e mensagens detalhadas de validações falhas do componente `@Valid`.
- **Testabilidade:** Criação de 6 classes de testes unitários utilizando JUnit 5 e Mockito, cobrindo de maneira isolada regras críticas como a ordem de escoamento FIFO de lotes, validações *multi-tenant*, lógica dos alertas de estoque e operações de CRUD padrão.

6 PADRÕES DE PROJETO GOF

6.1 Strategy (Comportamental)

- **Problema:** Evitar que a introdução de novas lógicas de alerta gere estruturas condicionais complexas (`if-else` ou `switch`), violando o princípio OCP.
- **Solução:** Criação de uma interface comum (*Strategy*) implementada de forma independente por cada regra de negócio, gerenciada por um mapa indexador (*Registry*).

```
// ESTRATEGIA: Interface comum para as regras de alerta
public interface StockAlertRule {
    AlertType type();
    List<AlertResponse> evaluate(Long companyId);
}

// CONCRETIZACAO 1: Regra para Produtos Vencendo
@Component
public class ExpiringStockRule implements StockAlertRule {
    @Override
    public AlertType type() { return AlertType.EXPIRING; }

    @Override
    public List<AlertResponse> evaluate(Long companyId) { // ...retorna os
        produtos proximos de vencer}
}

// CONCRETIZACAO 2: Regra para Estoque Baixo
@Component
public class LowStockRule implements StockAlertRule {
    @Override
    public AlertType type() { return AlertType.LOW_STOCK; }

    @Override
    public List<AlertResponse> evaluate(Long companyId) { // ...retorna os
        produtos com baixo estoque}
}

// CONTEXTO: Centraliza e delega a execucao dinamicamente atravs do Registry
@Service
public class AlertService {
    private final Map<AlertType, StockAlertRule> rulesByType;

    public AlertService(List<StockAlertRule> rules) {
        this.rulesByType =
```

```

        rules.stream().collect(Collectors.toMap(StockAlertRule::type,
        Function.identity()));
    }

    private List<AlertResponse> evaluate(AlertType type, Long companyId) {
        return rulesByType.get(type).evaluate(companyId);
    }
}

```

- **Benefício:** O `AlertService` torna-se totalmente extensível. A inclusão de um novo tipo de alerta exige apenas a criação de uma nova classe que implemente `StockAlertRule`, sem necessidade de modificar nenhuma linha de código do serviço central.

6.2 Adapter (Estrutural)

- **Problema:** Evitar que o núcleo de negócios e o serviço de domínio da aplicação fossem expostos ou rigidamente amarrados às particularidades de infraestrutura, tais como o protocolo de comunicação HTTP, uso do `RestTemplate`, strings de URLs e chaves JSON do microserviço externo em Flask.
- **Solução:** Aplicação do padrão *Adapter* em conformidade com as diretrizes de uma arquitetura hexagonal. Foi estabelecida uma interface abstrata de domínio que atua como uma porta de saída (*Port*). Em seguida, implementou-se um componente de infraestrutura (*Adapter*) responsável por interceptar as regras de negócio, realizar o mapeamento físico da requisição de rede HTTP via `RestTemplate` e consumir a Inteligência Artificial externa de forma totalmente transparente para o núcleo do sistema, conforme ilustrado abaixo:

```

// A PORTA: Abstracao de dominio independente de infraestrutura
public interface DemandForecastPort {
    List<AIPredictionResponse> forecast(List<AIDemandDataRequest> demandData);
}

// O ADAPTADOR: Encapsula as chamadas de rede HTTP e o RestTemplate
public class FlaskDemandForecastClient implements DemandForecastPort {
    @Override
    public List<AIPredictionResponse> forecast(List<AIDemandDataRequest>
        demandData) {
        ...
    }
}

```

```
// O CLIENTE DO PADRAO: Servico de Dominio acoplado apenas a Porta
public class AIService {
    private final DemandForecastPort demandForecastPort;

    public List<AIPredictionResponse> getPredictions() {
        return demandForecastPort.forecast(getDemandData());
    }
}
```

- **Benefícios:** Atua como uma robusta camada anticorrupção (*Anticorruption Layer*). Caso o microserviço em Flask seja substituído por um provedor em nuvem, por filas de mensageria ou por outra biblioteca de aprendizado profundo, o componente `AIService` permanecerá intacto. Além disso, a testabilidade é drasticamente otimizada, permitindo simular falhas e comportamentos da inteligência artificial por meio do isolamento de mocks da interface de modo trivial.

6.3 Builder (Criacional)

- **Problema:** A instanciação manual de entidades de domínio complexas contendo múltiplos atributos tornava os construtores ilegíveis e propensos a falhas de posicionamento de parâmetros.
- **Solução:** Acoplamento da anotação `@Builder` (Lombok) sobre as classes de entidade.
- **Benefícios:** Escrita fluente, legível e garantia de imutabilidade parcial durante as etapas de construção de dados nos serviços.
- **Exemplo de uso:**

```
Stock stock = Stock.builder()
    .quantity(dto.quantity())
    .expirationDate(dto.expirationDate())
    .product(product)
    .createdBy(authUser)
    .build();
```

7 DESIGN DE API

7.1 Estilo e justificativa

A API do Sched adota o estilo **REST** sobre HTTP/1.1, tráfegando exclusivamente JSON (`Content-Type: application/json`). A escolha justifica-se pela simplicidade de integração com o frontend web existente, pela maturidade do ecossistema Spring MVC para REST e pela natureza predominantemente CRUD do domínio. A especificação formal está no arquivo `docs/openapi.yaml` do repositório, no padrão **OpenAPI 3.0.3**.

7.2 Autenticação

Todas as rotas — exceto `POST /auth/company`, `POST /auth/login` e a própria especificação OpenAPI — exigem o header:

```
Authorization: Bearer <token>
```

O token JWT é obtido em `POST /auth/login` e assinado com HMAC256. Em caso de token ausente ou malformado a API retorna **401 Unauthorized**; em caso de token válido mas sem permissão para o recurso (ex.: usuário comum acessando rota de ADMIN ou dados de outra empresa), retorna **403 Forbidden**. Ambas as respostas seguem o schema `ErrorDetails`.

7.3 Versionamento

A versão corrente é a **v1**, implícita na raiz (`/`). A estratégia adotada é **versionamento por caminho de URL**: quando uma mudança incompatível for necessária, o prefixo `/api/v2/...` será introduzido, mantendo `/api/v1/...` ativo durante o período de transição. Até lá, a API evolui de forma retrocompatível, adicionando campos opcionais sem remover ou renomear os existentes.

7.4 Convenções de erro e paginação

Todos os erros retornam o schema `ErrorDetails`, definido como:

```
{ timestamp, status: integer, error: string, message: string | object
}
```

Em erros de validação de campos (`@Valid`), o campo `message` é um objeto chave→valor mapeando cada campo inválido à sua mensagem. Os códigos utilizados são:

- **400 Bad Request** — corpo inválido, campo faltante ou estoque insuficiente para a venda.

- **401 Unauthorized** — credenciais inválidas no login ou token ausente/expirado.
- **403 Forbidden** — token válido, mas sem permissão (papel insuficiente ou recurso de outra empresa).
- **404 Not Found** — recurso inexistente ou pertencente a outra empresa (isolamento multi-tenant).
- **409 Conflict** — violação de unicidade (ex.: CNPJ ou e-mail já cadastrado).

As listagens retornam o conjunto completo filtrado pela empresa do usuário autenticado. A paginação via `Pageable` do Spring Data está prevista como evolução obrigatória para `GET /stock` e `GET /sale`, registrada como dívida técnica.

7.5 Endpoints

A Tabela 1 apresenta todos os endpoints da API, organizados por grupo funcional.

Table 1: Endpoints da API Sched (OpenAPI 3.0.3).

Método	Rota	Descrição	Auth
<i>Auth</i>			
POST	/auth/company	Registra nova empresa e usuário ADMIN	Não
POST	/auth/login	Autentica e retorna token JWT	Não
POST	/auth/user	Cria usuário na empresa do ADMIN	ADMIN
<i>Usuário</i>			
GET	/user/me	Retorna o usuário autenticado	Sim
GET	/user	Lista usuários da empresa	ADMIN
GET	/user/{id}	Busca usuário por id	ADMIN
PUT	/user/{id}	Atualiza usuário	Sim
DELETE	/user/{id}	Exclusão lógica	ADMIN
<i>Empresa</i>			
GET	/company	Lista empresas ativas	ADMIN
GET	/company/{id}	Busca empresa por id	ADMIN
PUT	/company/{id}	Atualiza empresa	ADMIN
DELETE	/company/{id}	Exclusão lógica	ADMIN
<i>Produto</i>			
GET	/product	Lista produtos ativos da empresa	Sim
POST	/product	Cria produto	Sim
GET	/product/{id}	Busca produto por id	Sim
PUT	/product/{id}	Atualiza produto	Sim
DELETE	/product/{id}	Exclusão lógica (falha se há estoque ativo)	Sim
<i>Estoque</i>			
GET	/stock	Lista todos os lotes da empresa	Sim
GET	/stock/filterProduct	Resumo agregado por produto	Sim
POST	/stock/{id}	Cria lote para o produto id	Sim
PUT	/stock/{id}	Atualiza lote de estoque	Sim
DELETE	/stock/{id}	Remove lote de estoque	Sim
<i>Venda</i>			
GET	/sale	Lista vendas da empresa	Sim
POST	/sale/{id}	Registra venda do produto id (baixa FIFO)	Sim
<i>Alertas</i>			
GET	/alert/expiring	Produtos que vencem nos próximos 30 dias	Sim
GET	/alert/low-stock	Produtos com estoque baixo	Sim

7.6 Schemas principais

Os schemas de entrada e saída estão formalmente definidos no `openapi.yaml`. Os mais relevantes são descritos a seguir.

LoginRequest / LoginResponse: O login recebe `{email, password}` e retorna `{token: string}`, que deve ser enviado como `Bearer` em todas as chamadas subsequentes.

SaleRequest: Recebe apenas `{totalSold: integer}`. O serviço resolve internamente a baixa FIFO nos lotes, consumindo primeiramente os de vencimento mais próximo. Retorna `SaleResponse` com `totalPrice`, `totalSold`, `soldBy` e `saleDate`.

StockResponse: Consolida por produto os campos `availableQuantity`, `nextToExpireDate` e `latestExpirationDate`, permitindo ao frontend exibir o panorama de estoque sem chamadas adicionais.

AIPredictionResponse: Retorna por produto as previsões de demanda para 7, 15 e 30 dias (`prediction7Days`, `prediction15Days`, `prediction30Days`), a quantidade recomendada de reposição (`recommendedRestock`), um campo de alerta textual e os metadados do modelo utilizado (`modelUsed`, `modelMAE`).

ErrorDetails: Schema unificado para todos os erros, com `timestamp`, `status`, `error` e `message`. Em validações de campo, `message` é um objeto `{campo: mensagem}`; nos demais erros, é uma string descritiva.

8 DIAGRAMAS E MODELOS

8.1 Arquitetura geral do sistema

A Figura 1 apresenta a visão de componentes do sistema Sched no estilo C4 (nível de contêiner). O monolito Spring Boot organiza-se internamente em camadas (**controller**, **service**, **repository**, **domain**), com o pacote **security** interceptando todas as requisições antes de chegarem ao **controller**. O pacote **exception** (**GlobalExceptionHandler**) trata os erros em camada transversal. A única fronteira distribuída é o serviço de IA em Flask, consumido pelo adaptador **FlaskDemandForecastClient** via **DemandForecastPort**.

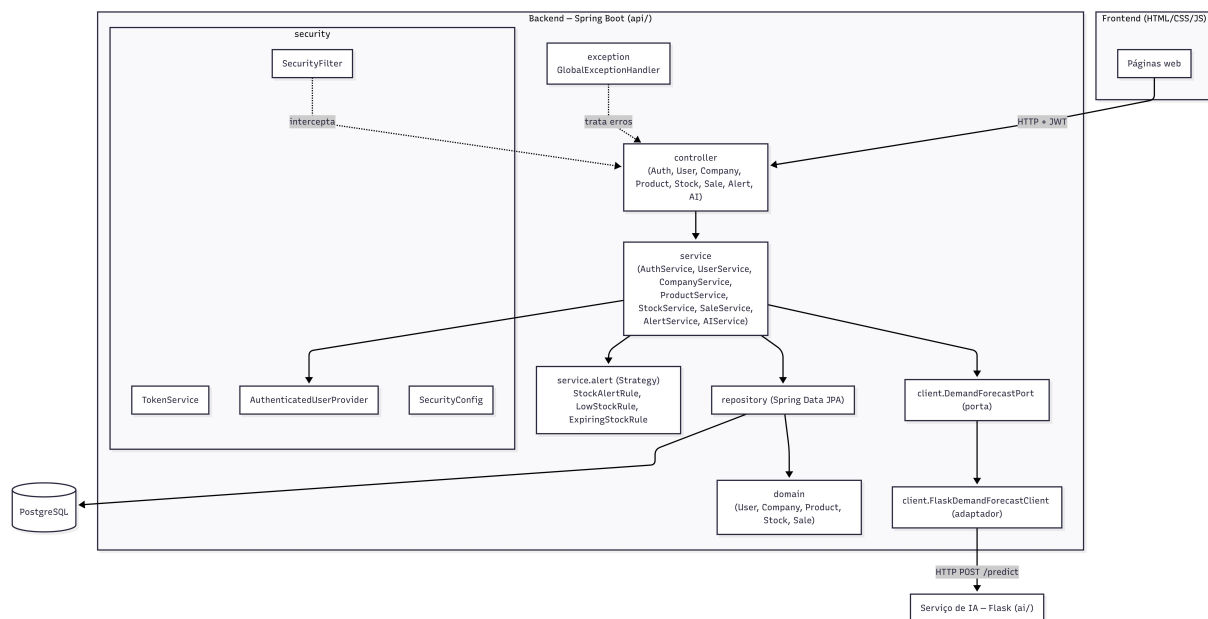


Figure 1: Arquitetura geral do sistema Sched — visão de componentes (estilo C4).

8.2 Diagramas de classes dos padrões GoF

8.2.1 Padrão Strategy — motor de alertas

A Figura 2 ilustra a aplicação do padrão Strategy. O **AlertService** agrega uma coleção de **StockAlertRule** por meio de um Registry (**Map<AlertType, StockAlertRule>**), selecionando a regra correta sem condicionais. As implementações **LowStockRule** e **ExpiringStockRule** são componentes independentes e substituíveis. O enum **AlertType** serve como chave de indexação do mapa.

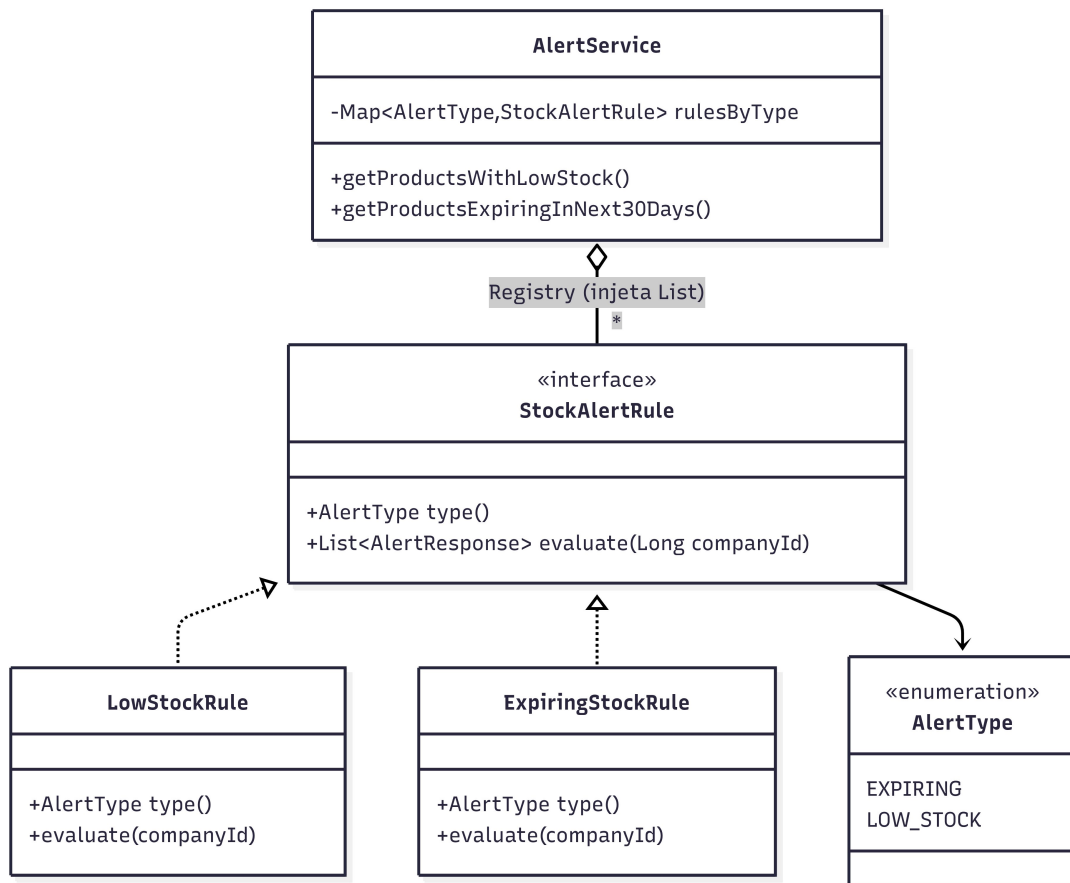


Figure 2: Diagrama de classes — padrão Strategy aplicado ao motor de alertas.

8.2.2 Padrão Adapter — integração com o serviço de IA

A Figura 3 ilustra o padrão Adapter (Port/Adapter). O `AIService` depende exclusivamente da interface `DemandForecastPort` (porta). O `FlaskDemandForecastClient` implementa essa porta encapsulando o `RestTemplate` e a URL do serviço Flask, funcionando como camada anticorrupção entre o domínio e a infraestrutura de rede.

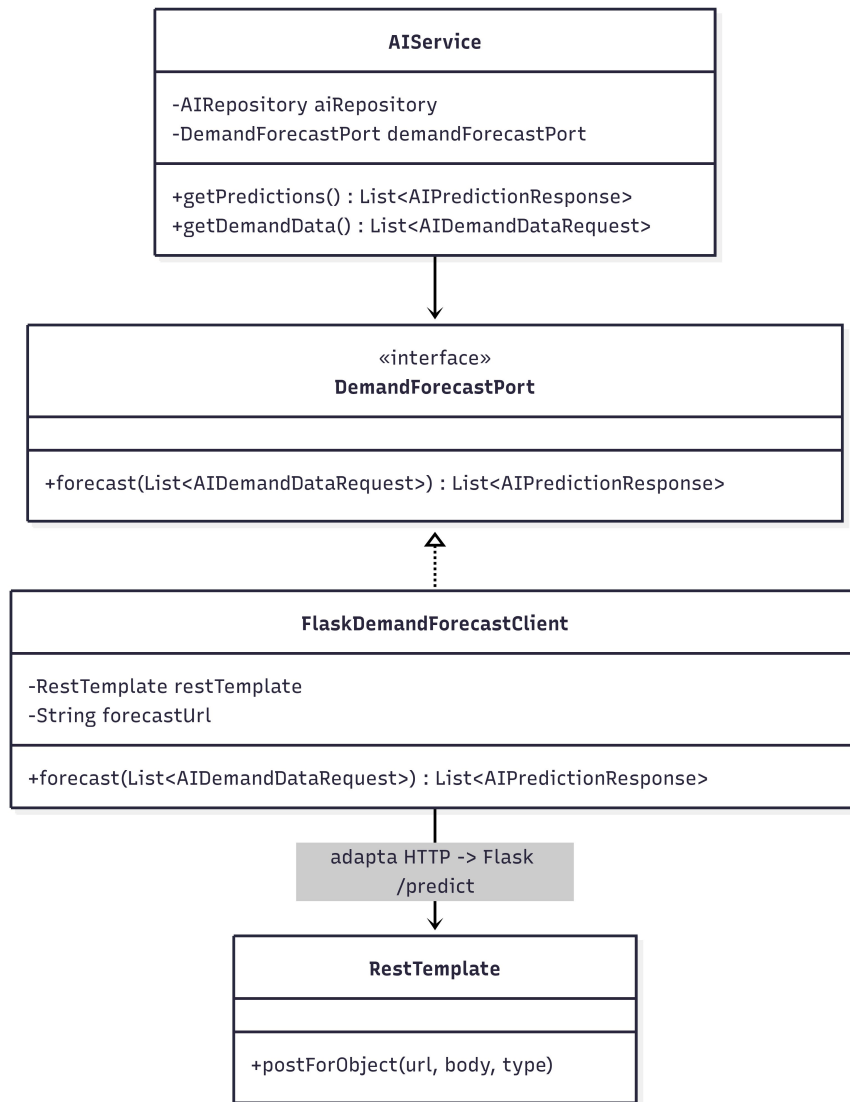


Figure 3: Diagrama de classes — padrão Adapter aplicado à integração com o serviço de IA.

8.2.3 Padrão Builder — construção de entidades de domínio

A Figura 4 ilustra o padrão Builder gerado pela anotação `@Builder` do Lombok. O **ProductService** utiliza o **ProductBuilder** para construir instâncias de **Product** de forma fluente, evitando construtores com múltiplos parâmetros posicionais e tornando o código de criação de entidades legível e resistente a erros de ordem de argumento.

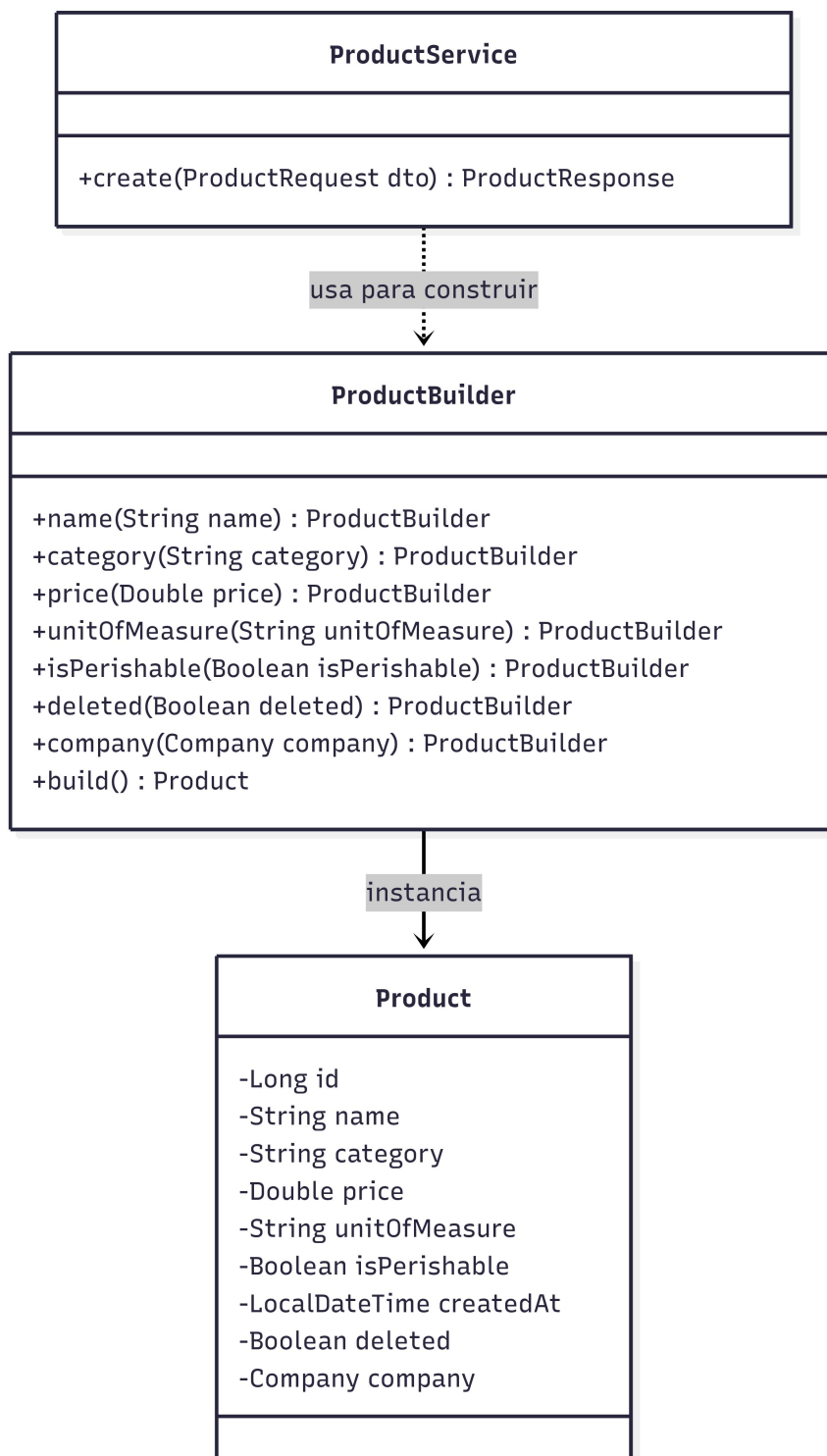


Figure 4: Diagrama de classes — padrão Builder aplicado à construção da entidade **Product**.

8.3 Diagrama de sequência — previsão de demanda por IA

A Figura 5 detalha o fluxo central de previsão de demanda (`GET /ai/predictions`). O frontend envia a requisição com token JWT; o `SecurityFilter` valida o token antes de o

AIController ser acionado. O AIService obtém o usuário autenticado via AuthenticatedUserProvider (passo 3–4), consulta os dados históricos no AIRepository (passo 5–6), delega a previsão à porta DemandForecastPort (passo 7), que é implementada pelo FlaskDemandForecastClient fazendo um HTTP POST /predict ao serviço Flask (passos 8–10). A resposta percorre o caminho inverso até retornar 200 OK com a lista de previsões ao frontend (passos 11–13).

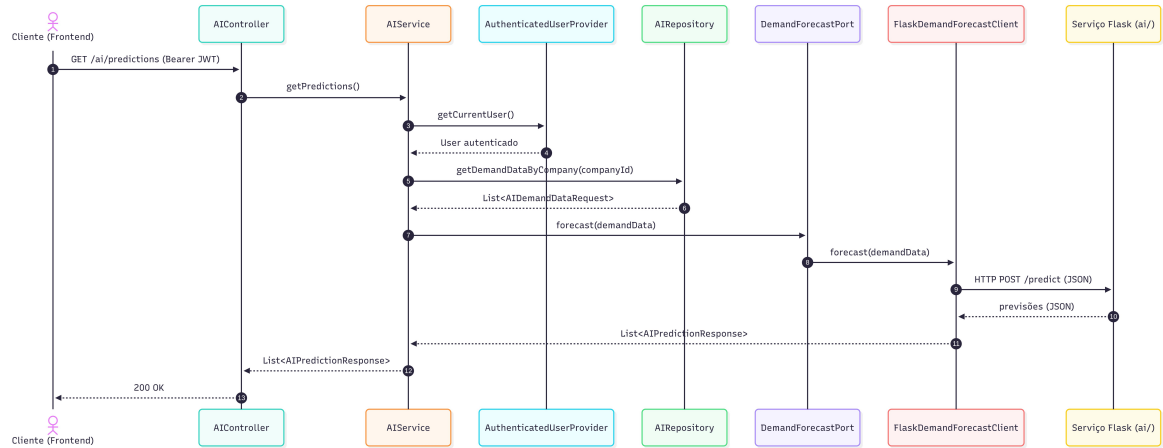


Figure 5: Diagrama de sequência — fluxo de previsão de demanda (GET /ai/predictions).

9 CONCLUSÃO

A refatoração incremental do backend do projeto Sched validou as premissas estabelecidas, preservando as regras de negócio legadas enquanto elevava a qualidade arquitetural do ecossistema de software. A explicitação de padrões consolidados de engenharia (como *Strategy*, *Adapter* e a inversão de dependências via portas de isolamento) proporcionou maior robustez contra alterações de código.

Vulnerabilidades de segurança críticas foram sanadas através da eliminação de chaves criptográficas em texto claro e fechamento de endpoints vulneráveis da inteligência artificial. A desacoplamento de chamadas estáticas do Spring Security aumentou drasticamente o índice de testabilidade unitária da aplicação.

Como limitações da iteração atual, destaca-se a ausência de mecanismos automáticos para invalidação de tokens JWT antes da janela de expiração (*blocklisting*), falta de paginação nativa nos endpoints de listagem de estoque e dependência de atualizações manuais no arquivo de contrato OpenAPI. Estas dívidas técnicas foram catalogadas e priorizadas como melhorias em ciclos futuros de engenharia, recomendando-se a introdução da biblioteca `springdoc-openapi` para geração automatizada de documentação e desenvolvimento de suítes de testes de integração *end-to-end*.

REFERÊNCIAS

FOWLER, M. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Design Patterns: Elements of Reusable Object-Oriented Software**. Boston: Addison-Wesley, 1994.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010:2023** — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. Genebra: ISO, 2023.

MARTIN, R. C. **Clean Code: A Handbook of Agile Software Craftsmanship**. Upper Saddle River: Prentice Hall, 2008.

MARTIN, R. C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Boston: Prentice Hall, 2017.

NYGARD, M. **Documenting Architecture Decisions**. 2011. Disponível em: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 5 jun. 2026.

ANEXOS

Os artefatos técnicos, diagramas e o código-fonte completo que fundamentam esta documentação de arquitetura encontram-se disponíveis publicamente no repositório Git listado a seguir. A estrutura de diretórios foi organizada de modo a garantir a rastreabilidade das decisões tomadas:

- **Repositório Principal do Projeto (Aberto e Acessível):**

Link para o código-fonte completo do ecossistema Sched.

<https://github.com/JheniferLais/TRABALHO-FINAL-ARQ>

- **Registro de Decisões Arquiteturais (/adrs):**

Diretório contendo todos os Registros de Decisões Arquiteturais (ADRs 001 a 008) mapeados individualmente em formato Markdown.

<https://github.com/JheniferLais/TRABALHO-FINAL-ARQ/tree/main/docs/adrs>

- **Fontes dos Diagramas Arquiteturais (/diagrams):**

Diretório contendo os arquivos de origem dos diagramas (modelados via ferramentas de diagramação como PlantUML/Mermaid).

<https://github.com/JheniferLais/TRABALHO-FINAL-ARQ/tree/main/docs/diagrams>

- **Instruções de Instalação e Execução (README.md):**

Documentação técnica principal localizada na raiz do repositório, contendo os pré-requisitos, instruções de build e comandos para execução local do monolito modular.

<https://github.com/JheniferLais/TRABALHO-FINAL-ARQ/blob/main/README.md>